

Universal Project Security Protocol

Protecting Any Project From Common Attack Vectors

Version 1.0

Prepared: July 18, 2026

Applies to: websites, APIs, internal tools, payments platforms, and AI-assisted ("vibecoding") projects

Universal Project Security Protocol

Protecting Any Project From Common Attack Vectors

Version 1.0 — July 2026

| Table of Contents

1. Executive Summary & Core Principle
 2. The Threat Landscape — How Projects Actually Get Hacked
 3. Secure Development Lifecycle Basics
 4. Authentication & Access Control
 5. Input Validation & Injection Prevention
 6. Data Protection & Encryption
 7. Secrets & Credential Management
 8. Payments & Financial Data Handling
 9. Network & Infrastructure Security
 10. Dependency & Supply Chain Security
 11. AI-Assisted Development (“Vibecoding”) Security
 12. Logging, Monitoring & Alerting
 13. Incident Response Runbook
 14. Backup & Disaster Recovery
 15. Compliance Basics (PCI DSS, GDPR, and When They Apply)
 16. Pre-Launch Security Checklist
 17. Ongoing Maintenance Cadence
 18. Appendix A — Command Cheat Sheet
 19. Appendix B — Incident Report Template
 20. Appendix C — Glossary
 21. Sign-Off
-

| 1. Executive Summary & Core Principle

No protocol guarantees a project will never be hacked. Anyone who tells you otherwise is selling something. Security is not a switch you flip to “safe” — it’s continuously reducing the number and size of the openings an attacker can use, and shortening how fast you notice and respond when one gets through anyway.

The realistic goal: make your project a harder, less rewarding target than the next one, catch problems early, and have a plan ready for when something does go wrong — because eventually, for any project handling real users or money, something will be attempted.

Core principle — Defense in Depth. No single control is enough on its own. Assume any one layer (a password, a firewall rule, a code review) will eventually fail or be bypassed, and stack multiple independent layers so one failure doesn’t mean total compromise.

How to use this document. This protocol is written to apply to *any* project — a website, an API, an internal tool, a payments platform. Not every section will be equally relevant to you; skim for what applies, but read Sections 2–7

regardless of project type, since they cover the most common real-world attack vectors.

2. The Threat Landscape — How Projects Actually Get Hacked

Understanding how real breaches happen matters more than any checklist. The overwhelming majority of successful attacks are not sophisticated zero-days — they exploit basic, well-known gaps:

- **Leaked credentials.** API keys or passwords committed to a public repo, pasted into a chat tool, or left in a config file that got exposed. This is consistently one of the top root causes of real breaches.
- **Broken access control.** An attacker changes an ID in a URL or API request (`/invoice/1002` → `/invoice/1003`) and can see or modify someone else's data because the server never checked ownership.
- **Injection attacks.** Untrusted input passed directly into a database query, shell command, or HTML output (SQL injection, command injection, XSS) — still extremely common despite being well understood for over two decades.
- **Outdated/vulnerable dependencies.** A known, publicly disclosed vulnerability in a library you're using, left unpatched.
- **Phishing / social engineering.** The attacker doesn't touch your code at all — they trick a person (you, a teammate, a support agent) into handing over credentials or approving something malicious.
- **Misconfiguration.** A cloud storage bucket left public, a debug mode left on in production, default admin credentials never changed.
- **Supply chain compromise.** A malicious or hijacked third-party package, browser extension, or CI/CD dependency injects malicious code into your build.
- **Prompt injection (new, AI-specific).** If you use an AI coding agent, malicious instructions hidden in content it reads can manipulate it into leaking data or running unintended commands (see Section 11).

Notice that most of this list is about **process and hygiene**, not exotic hacking techniques. That's good news: it means disciplined basics prevent the majority of real incidents.

3. Secure Development Lifecycle Basics

- **Shift security left.** Think about security when designing a feature, not after it ships. It's far cheaper to fix a design flaw on paper than in production.
 - **Threat-model new features briefly before building them:** who could misuse this, what's the worst case if they do, what stops them?
 - **Code review is non-negotiable**, including AI-generated code — a second set of eyes (even your own, later, with fresh focus) catches what one person misses.
 - **Separate environments.** Development, staging, and production should use different credentials, different databases, and ideally different cloud projects/accounts. Never let a dev/test build reach real production data.
 - **Version control everything**, including infrastructure config, so changes are traceable and reversible.
 - **Automate what you can** — tests, scans, and checks that run on every commit catch problems before a human has to remember to look.
-

4. Authentication & Access Control

This is where most real-world breaches of ordinary web apps happen — get this right and you eliminate a huge share of realistic risk.

- **Password storage:** hash with bcrypt, argon2, or scrypt — never plaintext, never reversible encryption, never a fast general-purpose hash like plain SHA-256/MD5.
- **Multi-factor authentication (MFA)** on any account with elevated privileges — admin panels, payment dashboards, cloud provider consoles, your own developer accounts.
- **Session management:** tokens should be long, random, expire after inactivity, and be invalidated on logout and password change.
- **Rate limiting and lockouts** on login, password reset, and any other endpoint an attacker could brute-force.
- **Server-side authorization checks on every request** — never trust a hidden form field, a client-side role flag, or “the button wasn’t shown” as your only protection. Check on the server: does *this* user actually own *this* resource?
- **Principle of least privilege** for both users and internal service accounts — nobody and nothing should have more access than their actual job requires.
- **Admin interfaces** should not be reachable from the open internet without additional protection (VPN, IP allow-list, or at minimum MFA + strong monitoring).

5. Input Validation & Injection Prevention

Treat every piece of data that comes from outside your own server code as hostile until proven otherwise — form fields, URL parameters, headers, file uploads, API payloads, even data from your own database if it originated from user input.

- **Parameterized queries / prepared statements** for all database access — never build SQL by string concatenation.

```
# Vulnerable
cursor.execute(f"SELECT * FROM users WHERE email = '{email}'")

# Safe
cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
```

- **Output encoding** for anything rendered into HTML, to prevent cross-site scripting (XSS). Most modern frameworks (React, Vue, Django templates) do this by default — don’t bypass it (e.g., avoid `dangerouslySetInnerHTML` unless the content is fully sanitized).
- **Validate file uploads by actual content**, not just file extension; store uploads outside the web root or in object storage (e.g., S3) with execution disabled.
- **Avoid building shell commands from user input.** If unavoidable, use safe subprocess APIs with argument arrays, never string-interpolated shell commands.
- **Validate on the server, always** — client-side validation is a UX nicety, not a security control, since it’s trivially bypassed.

6. Data Protection & Encryption

- **TLS everywhere.** No plaintext HTTP in production, ever — including internal service-to-service traffic where feasible. Enable HSTS.
- **Encrypt sensitive data at rest** — database-level or field-level encryption for anything sensitive (PII, financial data, health data).
- **Minimize what you store.** If you don’t need a piece of sensitive data, don’t collect or retain it — the safest data is

the data you never had.

- **Backups are also sensitive data.** Encrypt them, restrict access, and periodically test that a restore actually works (an untested backup is a hope, not a plan).
 - **Key management:** encryption keys should live in a dedicated secrets/key management service, not alongside the data they protect.
-

7. Secrets & Credential Management

- **No secrets in source code, ever** — not in a config file committed to git, not in a comment “temporarily,” not in a Slack message.
 - **Use a secrets manager:** cloud-native options (AWS Secrets Manager, GCP Secret Manager, Azure Key Vault) or third-party (1Password CLI, Doppler, HashiCorp Vault).
 - **Scan for leaked secrets before every commit** and periodically across your full git history (see Appendix A for exact commands) — a secret committed and later deleted is still exposed in history unless it’s rotated.
 - **Rotate immediately on any suspicion** of exposure — don’t wait to confirm; rotating is cheap, a breach is not.
 - **Distinct credentials per environment.** Dev, staging, and production should never share API keys, database passwords, or service account credentials.
 - **Least-privilege scoping** on every key and role — a key that can do everything is a much bigger loss if it leaks than one scoped narrowly.
-

8. Payments & Financial Data Handling

If your project processes payments, this section carries extra weight — both because payment systems are high-value targets and because there's real regulatory exposure (PCI DSS) if handled carelessly.

- **Never handle raw card numbers on your own servers if you can avoid it.** Use a processor's hosted/tokenized fields (Stripe Elements, PayPal Checkout, Braintree Drop-in) so card data flows directly from the customer's browser to the processor — it never touches your backend. This dramatically reduces your PCI DSS compliance burden (often down to the simplest self-assessment tier, SAQ A).
 - **Verify webhook signatures.** Payment providers notify you of events (payment succeeded, refund issued, subscription cancelled) via webhooks — always verify the cryptographic signature before trusting the payload, or an attacker can forge "payment succeeded" events.
 - **Use idempotency keys** on payment-creating requests so retries (network blips, double-clicks) can't cause duplicate charges.
 - **Reconcile regularly.** Periodically compare your internal records against the payment processor's records to catch discrepancies early.
 - **Log payment-related events without logging full card numbers, CVVs, or other prohibited data** — even in error logs or crash reports.
 - **Separate test and live API keys strictly**, and never let live keys exist anywhere outside production secrets management.
 - **If you ever do need to touch raw card data directly** (uncommon for most projects), that requires full PCI DSS compliance — a significantly heavier lift involving network segmentation, quarterly scans, and formal audits. Avoiding this entirely by using tokenization is almost always the right call for a project of any but the largest scale.
-

9. Network & Infrastructure Security

- **Firewall rules default to deny**, explicitly allow only what's needed (e.g., only ports 443/80 open publicly; database ports never exposed to the public internet).
 - **Keep servers and OS packages patched** on a regular schedule — most infrastructure compromises exploit known, already-patched vulnerabilities that simply weren't applied yet.
 - **Disable debug modes and verbose error pages in production** — stack traces and debug info are a gift to attackers mapping your system.
 - **Cloud storage buckets and databases:** verify they are not publicly readable/writable unless specifically intended to be, and double-check this after any infrastructure change.
 - **DDoS mitigation:** use a CDN/reverse proxy (Cloudflare, AWS CloudFront + Shield, etc.) in front of anything public-facing, both for performance and basic attack absorption.
 - **Container security:** scan container images for known CVEs before deployment; don't run containers as root unless required.
-

10. Dependency & Supply Chain Security

- **Audit dependencies regularly**, not just at project start — new CVEs are disclosed in existing packages constantly.
- **Verify new packages before adding them:** correct name (watch for typosquats like `requeusts` vs `requests`), active maintenance, plausible download counts, no unusual install scripts.
- **Pin versions for security-critical libraries** (auth, crypto, payment SDKs) rather than always pulling "latest," so upgrades are deliberate, reviewed decisions.
- **Automate scanning in CI** so a vulnerable dependency is flagged before merge, not discovered after a breach.

- **Review third-party integrations' permissions** (OAuth scopes, API access levels) — grant only what's actually needed.
-

11. AI-Assisted Development (“Vibecoding”) Security

If you’re using an AI coding agent (e.g., Google Antigravity, or similar tools) to build any part of this project, layer these controls on top of everything above — agentic IDEs have their own, newer attack surface:

- **Disable auto-run/auto-approve of agent commands.** Require human confirmation before the agent executes anything beyond read-only actions.
 - **Treat all content the agent reads from the web or third-party files as untrusted** — indirect prompt injection (malicious instructions hidden in a webpage or document) is a real, disclosed risk class in current agentic IDEs, capable of tricking the agent into leaking secrets or running unintended commands.
 - **Never leave production credentials accessible to an agent session** — scope what the agent’s environment can reach to the minimum needed.
 - **Diff and review every AI-generated change** before merging — don’t “accept all.”
 - **Run the same dependency and secret scans on AI-generated code** as on human-written code; the agent can add a vulnerable or typosquatted package just as easily as a person can.
 - **Verify agent settings actually work** — e.g., in a fresh session, confirm the agent can’t run a command without approval, and can’t read your `.env` file when asked to.
-

12. Logging, Monitoring & Alerting

You can’t respond to what you don’t notice — this is the layer most small projects skip, and the one that most extends how long a breach goes undetected.

- **Log authentication events:** logins, failed logins, password resets, privilege changes.
 - **Log administrative actions:** who changed what, when — especially for anything touching money, permissions, or user data.
 - **Never log secrets, full card numbers, or full session tokens.**
 - **Set up basic alerting** for anomalies: a spike in failed logins, an unusual spike in data export/download volume, requests from unexpected geographies for an otherwise regional product.
 - **Centralize logs** somewhere an attacker who compromises the main server can’t simply delete them (a separate logging service or write-once storage).
-

13. Incident Response Runbook

When something looks wrong — unexpected charges, unfamiliar admin activity, a report from a user, a security scanner alert — work through this in order:

1. **Stay calm and stop the bleeding first.** Revoke/rotate the specific credential or session involved before doing anything else.
 2. **Contain.** Disable the affected account, endpoint, or server if actively being exploited; take clearly compromised systems offline rather than trying to “clean” them live.
 3. **Rotate broadly.** Rotate every credential that could plausibly have been exposed, not just the one you’re sure about.
 4. **Investigate.** Check logs for the entry point and scope: what was accessed, when, from where. Diff recent code/config changes for anything unauthorized.
 5. **Assess who’s affected.** Determine if user data, payment data, or shared infrastructure was touched — this determines your notification obligations (see Section 15).
 6. **Notify as required.** Legal obligations for breach notification vary by jurisdiction and data type — when in doubt on payment or personal data exposure, consult a lawyer promptly; timelines are often short.
 7. **Fix the root cause,** not just the symptom — patch the actual vulnerability, not only the specific exploited instance.
 8. **Post-mortem.** Document what happened (Appendix B template) and update this protocol with anything it missed.
-

14. Backup & Disaster Recovery

- **Automated, regular backups** of databases and critical files — frequency matched to how much data loss you could tolerate.
 - **Encrypted backups, access-restricted** separately from production credentials, so a compromised production system doesn’t automatically compromise backups too.
 - **Test restores periodically** — an untested backup is unverified until you’ve actually restored from it successfully.
 - **Off-site/off-account backup copies** so a single compromised cloud account can’t destroy both production and backups simultaneously.
 - **Document the recovery procedure** so it doesn’t have to be improvised under pressure during an actual incident.
-

15. Compliance Basics (PCI DSS, GDPR, and When They Apply)

- **PCI DSS** applies if you handle, store, process, or transmit payment card data. Using tokenized/hosted payment fields (Section 8) typically qualifies you for the lightest self-assessment tier (SAQ A) rather than a full audit — this is usually the single highest-leverage compliance decision you can make.
- **GDPR** (or similar regional privacy laws) applies if you handle personal data of users in the relevant jurisdiction (e.g., EU residents), regardless of where your company is based — covers consent, data minimization, the right to deletion, and breach notification timelines.
- **This document is not legal advice.** Compliance obligations depend on your specific jurisdiction, data types, and business model — consult a lawyer or compliance specialist before launch if you're handling payments or personal data at any meaningful scale.

16. Pre-Launch Security Checklist

Work through this before taking any project live, regardless of type:

- ☐ All secrets in a secrets manager, none in source code or committed history
- ☐ MFA enabled on all admin/cloud provider accounts
- ☐ Server-side authorization checks verified on every sensitive endpoint (not just hidden UI)
- ☐ Parameterized queries used throughout; no string-built SQL
- ☐ Dependency audit run, all high/critical issues resolved
- ☐ Secret-scanning run across full git history
- ☐ HTTPS/TLS enforced everywhere, HSTS enabled
- ☐ Debug mode and verbose error pages disabled in production
- ☐ Rate limiting on login, password reset, and public API endpoints
- ☐ Payment integration uses tokenized/hosted fields, not raw card handling
- ☐ Webhook signature verification implemented and tested
- ☐ Backups configured, encrypted, and a test restore has been performed
- ☐ Logging and basic alerting in place for auth events and anomalies
- ☐ Incident response contacts and rotation procedure documented
- ☐ Penetration test or automated scan (e.g., OWASP ZAP) run against staging
- ☐ Attempted to access another user's data by manipulating IDs/parameters — confirmed blocked

17. Ongoing Maintenance Cadence

Security is not a one-time setup — treat this as a recurring commitment:

Task	Frequency
Dependency/CVE audit	Before every deploy, minimum monthly
Secret scan across repo	Weekly
Review access/IAM permissions	Monthly
Rotate credentials touched by any suspicious activity	Immediately
Full backup restore test	Quarterly
Review this protocol for gaps	Quarterly, or after any incident
Dependency version pin review	Quarterly

18. Appendix A — Command Cheat Sheet

Secret scanning:

```
gitleaks detect --source . --verbose
trufflehog git file://. --only-verified
```

Dependency auditing:

```
npm audit --audit-level=high      # Node.js
pip-audit                         # Python
trivy image your-image:tag        # container images
```

Static analysis:

```
semgrep --config=auto .
```

Basic automated web scan:

```
# OWASP ZAP baseline scan against a staging URL
docker run -t owasp/zap2docker-stable zap-baseline.py -t https://staging.yourproject.com
```

Check for open ports on a server:

```
sudo ss -tulpn
```

19. Appendix B — Incident Report Template

INCIDENT REPORT

Date/time detected:

Detected by:

System/project affected:

What happened (factual, no speculation):

How it was detected:

Immediate actions taken (stop / contain / rotate):

Credentials rotated:

Root cause (best understanding):

Was user data or payment data affected? Y / N — details:

Notification obligations assessed? Y / N — action taken:

Changes made to prevent recurrence:

Protocol updated? Y / N — section(s) affected:

20. Appendix C — Glossary

Term	Meaning
Broken access control	A flaw where the server fails to verify a user is authorized to access/modify a specific resource
Injection attack	Untrusted input executed as code or commands (SQL injection, command injection, XSS)
Idempotency key	A unique identifier attached to a request so retries don't cause duplicate effects (e.g., double charges)
Tokenization	Replacing sensitive data (like a card number) with a non-sensitive reference token, so the real data never touches your systems
PCI DSS	Payment Card Industry Data Security Standard — the compliance framework for handling card data
Prompt injection	Malicious instructions hidden in content an AI agent reads, causing it to act against the user's actual intent
Least privilege	Granting only the minimum access needed for a task or role
Defense in depth	Layering multiple independent security controls so a single failure doesn't cause total compromise
SAQ A	The simplest PCI DSS self-assessment tier, available when card data never touches your own servers

21. Sign-Off

This protocol is a living document. Update it whenever your project's scope changes, a new vulnerability class becomes relevant, or an incident teaches you something this version missed. Treat Section 17's maintenance cadence as a standing commitment, not a one-time task.

Reviewed by: _____ **Date:** _____

Next review due: _____